# Introduction to Python

# Python Interpreter

```
C:\>
C:\>python
Python 2.7.13 ...
Type "help", "copyright", ...
>>>

>>>
```

Interactive mode

# Python Interpreter

```
>>>123
123
>>>`program'
program
>>>help()
Welcome to Python 2.7! ...
...
```

# Standard Data Types

- Numbers
- String
- List
- Tuple
- Dictionary

# Arithmetic Operators

```
>>>10+5
15
>>>5*2
10
>>>5/2
2
```

```
>>>5.5//2
2.0
>>>10%3
1
>>>5**2
25
```

# Comparison Operators

```
>>>2.0==2
True
>>>'ab'<'d'
True
>>>5%2==0
False
```

Relational operators

| | | |
|---|---|---|
| < | > | <= |
| >= | == | != |

Logical operators

*and* *or* *not*

# Variable Assignments

```
>>>a=123
>>>a=b=c=5
>>>x=‘a string’
>>>m,n = 456, ‘var’
>>>a = 5==6
>>>a
False
```

# Lists

Ordered Collection of data objects

```
>>>myList = [17, True, 'hello']
>>>len(myList)
3
>>>myList[1] = 45
>>>del myList[0]
>>>myList
[45, 'hello']
```

# Indexing & Slicing

| | | | | | |
|---|---|---|---|---|---|
| s | t | r | i | n | g |

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| -6 | -5 | -4 | -3 | -2 | -1 | |

# Indexing & Slicing

```
>>>a = range(10)
>>>a[2:5]
[2, 3, 4, 5]
>>>a[5:]
[5, 6, 7, 8, 9]
>>>a[:3]
[0, 1, 2]
```

```
>>>a[-3]
7
>>>a[-6 : -3]
[4, 5, 6]
>>>a[: : 2]
[0, 2, 4, 6, 8]
>>>a[: : -1]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

# More on lists ...

```
list.append(x)
list.extend(L)
list.insert(i,x)
list.remove(x)
list.index(x)
list.count(x)
```

```
list.sort()

list.reverse(x)
list.sort(reverse=True)
```

# More on lists ...

```
>>>[1, 2, 3] + [6, 7, 8]
[1, 2, 3, 6, 7, 8]
```

concatenation

```
>>>a = [0]*6
>>>a
[0, 0, 0, 0, 0, 0]
```

repetition

# Input

```
>>>a = raw_input('--> ')
--> 123
>>>a
'123'
>>>a = int(a)
123
```

User input

Type conversion

# Input

```
>>>a = input('--> ')
--> 4+5
>>>a
9
```

evaluate input

# Output

```
>>>print 'Hello!!'
Hello!!
>>>a = '123'; x = 2
>>>print  a, x
123 2
```

# Formatted Output

```
>>>'%s %d' % ('abc', 123)
'abc 123'
>>>'{}_{}' % ('abc', 123)
'abc_123'
>>>'{1}_{0}' % ('abc', 123)
'123_abc'
>>>'{:d} {:f}' % ('abc', 123)
'123 123.000000'
```

# Code Blocks

Block 1

Block 2

Block 3

Block 2, continuation

Block 1, continuation

# Control Structures

```
if score >= 90:
    print('A')
else:
    if score >=80:
        print('B')
    else:
        if score >= 70:
            print('C')
        else:
            print('F')
```

```
if score >= 90:
    print('A')
elif score >=80:
    print('B')
elif score >= 70:
    print('C')
else:
    print('F')
```

# Control Structures

```
>>>count = 1
>>>while count < 3:
...     print count
...     count += 1
...
1
2
>>>
```

Compound

```
while count < 5 and not done:
...
```

# Control Structures

```
>>>for item in [1, 4, 5, 6]:
...     print item
...
1
4
5
6
>>>
```

```
>>>x = [1, 4, 5, 6]
>>>for item in x:
...
```

# Control Structures

```
>>>for  x in range(2,5):
...      print x**2
...
4
9
16
>>>
```

# List Comprehensions

```
>>>a = [ ]
>>>for  x in range(2,5):
. . .     a.append(x**2)
. . .
>>>print a
[4, 9, 16]
```

```
>>>a = [x**2 for x in  \
. . .            range(2,5) ]
>>>print a
[4, 9, 16]
```

# List Comprehensions

```
>>> [ch.upper() for ch in 'comprehension' \
. . .   if ch not in 'aeiou']
['C', 'M', 'P', 'R', 'H', 'N', 'S', 'N']
```

# List Comprehensions

```
>>> a = [x**2 for x in range(10)]
>>> y = [x for x in S if x % 2 == 0]
>>> print a; print y
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
[0, 4, 16, 36, 64]
```

# Input

```
>>> a = raw_input('')
4 5 16 20
>>> a
'4 5 16 20'
>>> a = a.split()
>>> a
['4', '5', '16', '20']
```

```
>>> b = [ ]
>>> for x in a :
. . .     b.append( int(x) )
. . .
>>> b
[4, 5, 16, 20]
```

# Input

```
>>> a = [int(x) for x in a ]
>>> a
[4, 5, 16, 20]
```

```
>>> b = [ ]
>>> for x in a :
...     b.append( int(x) )
...
>>> b
[4, 5, 16, 20]
```

# Input

```
>>> a = raw_input()
4 15 16 20
>>> a = a.split()
>>> a = [int(x) for x in a ]
>>> a
[4, 5, 16, 20]
```

```
>>> a = map( int, a)
>>> a
[4, 5, 16, 20]
```

# Input

```
>>> a = raw_input()
4 15 16 20
>>> a = a.split()
>>> a = map( int, a)
>>> a
[4, 5, 16, 20]
```

```
>>> a = map( int, \
. . .   raw_input().split() )
4 15 16 20
>>> a
[4, 5, 16, 20]
```

# List Comprehension

```
>>> words = 'The quick brown fox \
. . . jumps over the lazy dog'.split()
>>> print words
['The', 'quick', 'brown', 'fox', 'jumps',
'over', 'the', 'lazy', 'dog']
```

# List Comprehension

```
>>> stuff = [[w.upper(), w.lower(),\
...                len(w)] for w in words]
>>> for i in stuff:
...     print i
...
```

# List Comprehension

```
['THE', 'the', 3]
['QUICK', 'quick', 5]
['BROWN', 'brown', 5]
['FOX', 'fox', 3]
['JUMPS', 'jumps', 5]
['OVER', 'over', 4]
['THE', 'the', 3]
['LAZY', 'lazy', 4]
['DOG', 'dog', 3]
```

# Functions

```
>>> def square(x):
...         return x**2
...
>>> square(4)
16
>>> square( square( 4 ) )
256
>>>
```

# Functions

```
>>> def myFun( x ):
...     x.append([1,2,3])
>>>
```

```
>>> a = range(5)
>>> myFun( a )
>>> a
[0, 1, 2, 3, 4, [ 1, 2, 3] ]
```

Variables are passed by reference

# Functions

```
>>> def printInfo( name, age = 20):
. . .     print 'Name: ',name
. . .     print 'Age: ',age
. . .
>>>
```

```
>>> printInfo( "Ram")
Name: Ram
Age: 20
>>> printInfo( age = 25, \
. . .   name = 'Shyam')
Name: Shyam
Age: 25
>>>
```

# Lambda Functions

```
>>> stuff = [[w.upper(), w.lower(),\
...                len(w)] for w in words]
>>> for i in stuff:
...     print i
...
```

```
['THE', 'the', 3]
['QUICK', 'quick', 5]
['BROWN', 'brown', 5]
['FOX', 'fox', 3]
['JUMPS', 'jumps', 5]
['OVER', 'over', 4]
['THE', 'the', 3]
['LAZY', 'lazy', 4]
['DOG', 'dog', 3]
```

# Lambda Functions

```
>>> stuff = [[w.upper(), w.lower(),\
. . .                len(w)] for w in words]
>>> stuff = map( . . . ,   words )
```

```
lambda w: [w.upper(), w.lower(), len(w)]
```

# File I/O

```
>>> fin = open( 'words.txt', 'r')
>>> for line in fin:
. . .       print line
. . .
>>>
>>> fin.close()
```

# File I/O

```
>>> fout = open( 'output.txt', 'w')
>>> fout.write( string )
>>>
>>> fout.close()
```

# Dictionaries

```
{‘key’: ‘value’}
```

# Dictionaries

```
>>> lang = { }
>>>
>>> lang = { 'germany': german, 'france': french}
>>> lang[ 'germany' ]
'german'
>>> lang[ 'japan' ] = 'japanese'
```

# Dictionaries

```
>>> lang[ ‘usa’ ]
KeyError: ‘usa’
>>>
>>> lang.get( ‘usa’, default = ‘english’ )
‘english’
```

# Dictionaries

```
>>> for key in lang:
. . .      print key
. . .
japan
germany
france
>>>
```

```
>>> lang.keys()
['germany', 'japan', 'france']
>>> lang.values()
['japanese', 'german', 'french']
```

# Dictionaries

```
>>> lang.items()
[('japan', 'japanese'), ('germany', 'german'), ('france',
'french')]
>>> for k,v in lang.items():
. . .      print  k,'>',v
. . .
japan > japanese
germany > german
france > french
```

# Dictionaries

```
>>> for key in sorted( lang.keys() ):
. . .       print key, lang[key]
. . .
>>> del lang[‘japan']; # remove entry with key ‘japan'
>>> lang.clear();     # remove all entries in lang
>>> del lang ;        # delete entire dictionary
```

# Few more things ...

```
>>> s = ‘   abc   ‘
>>> s.rstrip( )
‘   abc’
>>> s.lstrip( )
‘abc   ‘
>>> s.strip()
‘abc’
```

# Few more things ...

```
>>> x = [ 'a', 'b', 'c' ]
>>> '+'.join( x )
'a+b+c'
>>>
```

# Few more things ...

```
>>> a = 2
>>> b = 5
>>> a, b = b, a
>>> a
5
>>> b
2
```

# Few more things ...

## Tuples

```
>>> x = ( 2017, 'str', 105)
>>> x[0]
2017
>>> x[0] = 2020
TypeError: . . .
```

Values cannot be changed

# Few more things ...

## Sets

```
>>> x = [ 1, 5, 3, 7, 2, 5, 1, 1]
>>> set( x )
set([ 1, 2, 3, 5, 7])
>>> x[0]
TypeError: . . .
>>> set.add(10)
```

Stores unique values

Does not support indexing

# Few more things ...

**Sets**

x in s

s.union(t)

s.intersection(t)

s.difference(t)

# Compiling a File

```
C:\> python file.py

C:\> python file.py < in.txt

C:\> python file.py < in.txt > out.txt
```